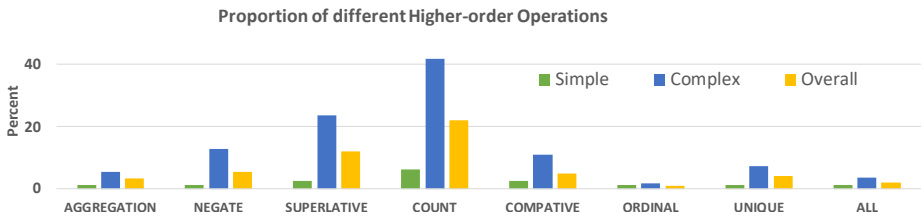

Figure 2: Proportion of different higher-order operations from the simple/complex channels.

Channel	#Sentence	#Table	Len(Ent)	Len(Ref)	Split	#Sentence	Table	Row	Col
Simple	50,244	9,189	13.2	13.1	Train	92,283	13,182	14.1	5.5
Complex	68,031	7,392	14.2	14.2	Val	12,792	1,696	14.0	5.4
Total	118,275	16,573	13.8	13.8	Test	12,779	1,695	14.2	5.4

Table 1: Basic statistics of the data collected from the simple/complex channel and the division of Train/Val/Test Split in the dataset, where "Len" denotes the averaged sentence length.

2.3 Dataset Statistics

Inter-Annotator Agreement: After the data collection pipeline, we merged the instances from two different channels to obtain a diverse yet clean dataset for table-based fact verification. We sample 1000 annotated (table, statement) pairs and re-distribute each to 5 individual workers to re-label them as either `ENTAILED` or `REFUTED`. We follow the previous works (Thorne et al., 2018; Bowman et al., 2015) to adopt the Fleiss Kappa (Fleiss, 1971) as an indicator, where Fleiss $\kappa = \frac{\bar{p_c}-\bar{p_e}}{1-\bar{p_e}}$ is computed from from the observed agreement $\bar{p_c}$ and the agreement by chance $\bar{p_e}$. We obtain a Fleiss $\kappa = 0.75$, which indicates strong inter-annotator agreement and good-quality.

Dataset Statistics: As shown in Table 1, the amount of data harvested via the complex channel slightly outnumbers the simple channel, the averaged length of both the positive and negative samples are indistinguishable. More specifically, to analyze to which extent the higher-order operations are included in two channels, we group the common higher-order operations into 8 different categories. As shown in Figure 2, we sample 200 sentences from two different channels to visualize their distribution. We can see that the complex channel overwhelms the simple channel in terms of the higher-order logic, among which, count and superlatives are the most frequent. We split the whole data roughly with 8:1:1 into train, validation[7], and test splits and shows their statistics in Table 1. Each table with an average of 14 rows and 5-6 columns corresponds to 2-20 different statements, while each cell has an average of 2.1 words. In the training split, the positive instances slightly outnumber the negative instances, while the validation and test split both have rather balanced distributions over positive and negative instances.

3 Models

With the collected dataset, we now formally define the table-based fact verification task: the dataset is comprised of triple instances $(\mathbf{T}, S, L)$ consisting of a table $\mathbf{T}$, a natural language statement $S = s_1, \cdots, s_n$ and a verification label $L \in \{0, 1\}$. The table $\mathbf{T} = \{T_{i,j} | i \leq R_T, j \leq C_T\}$ has R_T rows and C_T columns with the T_{ij} being the content in the (i, j)-th cell. T_{ij} could be a word, a number, a phrase, or even a natural language sentence. The statement S describes a fact to be verified against the content in the table $\mathbf{T}$. If it is entailed by $\mathbf{T}$, then $L = 1$, otherwise the label $L = 0$. Figure 1 shows some entailed and refuted examples. During training, the model and the learning algorithm are presented with K instances like $(\mathbf{T}, S, L)_{k=1}^{K}$ from the training split. In the testing stage, the model is presented with $(\mathbf{T}, S)_{k=1}^{K'}$ and supposed to predict the label as $\hat{L}$. We measure the performance by the prediction accuracy $Acc = \frac{1}{K'} \sum_{1}^{K'} \mathbb{I}(\hat{L}_k = L_k)$ on the test set. Before building the model, we first perform entity linking to detect all the entities in the statements. Briefly, we first lemmatize the words and search for the longest sub-string matching pairs between statements and table cells/captions, where the matched phrases are denoted as the linked entities. To focus on statement verification against the table, we do not feed the caption to the model and simply

[7] We filter roughly 400 sentences from abnormal tables including hyperlinks, math symbols, etc

mask the phrases in the statements which link to the caption with placeholders. The details of the entity linker are listed in the Appendix. We describe our two proposed models as follows.

3.1 LATENT PROGRAM ALGORITHM (LPA)

In this approach, we formulate the table fact verification as a program synthesis problem, where the latent program algorithm is not given in TABFACT. Thus, it can be seen as a weakly supervised learning problem as discussed in Liang et al. (2017); Lao et al. (2011). Under such a setting, we propose to break down the verification into two stages: (i) latent program search, (ii) discriminator ranking. In the first program synthesis step, we aim to parse the statement into programs to represent its semantics. We define the plausible API set to include roughly 50 different functions like *min, max, count, average, filter, and* and realize their interpreter with Python-Pandas. Each API is defined to take arguments of specific types (*number, string, bool, and view (e.g sub-table)*) to output specific-type variables. During the program execution, we store the generated intermediate variables to different-typed caches $\mathcal{N}, \mathcal{R}, \mathcal{B}, \mathcal{V}$ (Num, Str, Bool, View). At each execution step, the program can fetch the intermediate variable from the caches to achieve semantic compositionality. In order to shrink the search space, we follow NSM (Liang et al., 2017) to use trigger words to prune the API set and accelerate the search speed. The definitions of all API, trigger words can be found in the Appendix. The comprehensive the latent program search procedure is summarized in Algorithm 1,

Algorithm 1 Latent Program Search with Comments

```
Initialize Number Cache N, String Cache R, Bool Cache B, View Cache V → ∅
Push linked numbers, strings from the given statement S into N, R, and push T into V
Initialize the result collector P → ∅ and an empty program trace P = ∅
Initialize the Queue Q = [(P, N, R, B, V)], we use Q to store the intermediate states
Use trigger words to find plausible function set F, for example, more will trigger Greater function.
while loop over time t = 1 → MAXSTEP do:
    while (P, N, R, B, V) = Q.pop() do:
        while loop over function set f ∈ F do:
            if arguments of f are in the caches then
                Pop out the required arguments arg_1, arg_2, ··· , arg_n for different cachess.
                Execute A = f(arg_1, ··· , arg_n) and concatenate the program trace P.
                if Type(A)=Bool then
                    if N = S = B = ∅ then
                        P.push((P, A)) # The program P is valid since it consumes all the variables.
                        P = ∅ # Collect the valid program P into set P and reset P
                    else
                        B.push(A) # The intermediate boolean value is added to the bool cache
                        Q.push((P, N, R, B, V)) # Add the refreshed state to the queue again
                if Type(A) ∈ {Num, Str, View} then
                    if N = S = B = ∅ then
                        P = ∅;break # The program ends without consuming the cache, throw it.
                    else
                        push A into N or S or V # Add the refreshed state to the queue for further search
                        Q.push((P, N, R, B, V))
Return the triple (T, S, P) # Return (Table, Statement, Program Set)
```

and the searching procedure is illustrated in Figure 3.

After we collected all the potential program candidates $\mathcal{P} = \{(P_1, A_1), \cdots, (P_n, A_n)\}$ for a given statement S (where (P_i, A_i) refers to i-th candidate), we need to learn a discriminator to identify the "appropriate" traces from the set from many erroneous and spurious traces. Since we do not have the ground truth label about such discriminator, we use a weakly supervised training algorithm by viewing all the label-consistent programs as positive instances $\{P_i|(P_i, A_i); A_i = L\}$ and the label-inconsistent program as negative instances $\{P_i|(P_i, A_i); A_i \neq L\}$ to minimize the cross-entropy of discriminator $p_\theta(S, P)$ with the weakly supervised label. Specifically, we build our discriminator with a Transformer-based two-way encoder (Vaswani et al., 2017), where the statement encoder encodes the input statement S as a vector $Enc^S(S) \in \mathbb{R}^{n \times D}$ with dimension D, while the program encoder encodes the program $P = p_1, \cdots, p_m$ as another vector $Enc^P(P) \in \mathbb{R}^{m \times D}$, we concatenate these two vectors and feed it into a linear projection layer

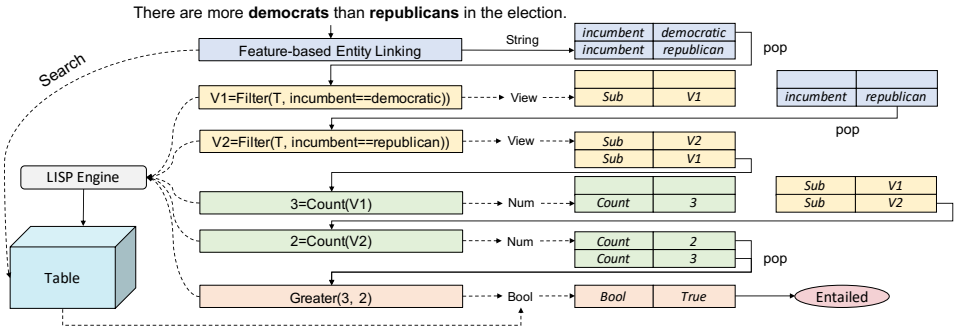

Figure 3: The program synthesis procedure for the table in Figure 1. We link the entity (e.g. *democratic*, *republican*), and then composite functions on the fly to return the values from the table.

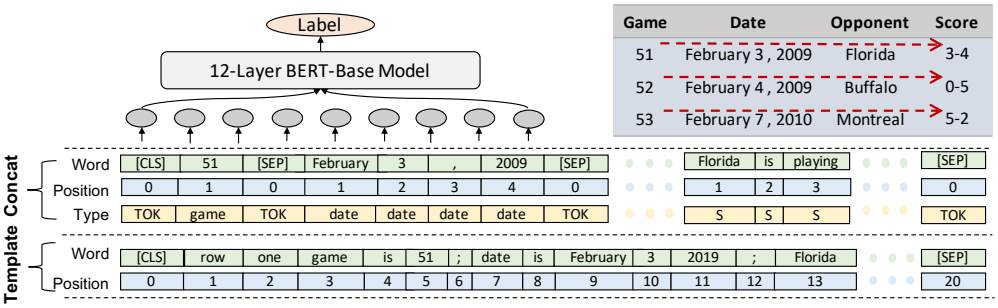

Figure 4: The diagram of Table-BERT with horizontal scan, two different linearizations are depicted.

to compute $p_\theta(S, P) = \sigma(v_p^T[Enc^S(S); Enc^P(P)])$ as the relevance between S and P with weight $v_p \in \mathbb{R}^D$. At test time, we use the discriminator p_θ to assign confidence $p_\theta(S, P)$ to each candidate $P \in \mathcal{P}$, and then either aggregate the prediction from all hypothesis with the confidence weights or rank the highest-confident hypothesis and use their outputs as the prediction.

3.2 TABLE-BERT

In this approach, we view the table verification problem as a two-sequence binary classification problem like NLI or MPRC (Wang et al., 2018) by linearizing a table $\mathbf{T}$ into a sequence and treating the statement as another sequence. Since the linearized table can be extremely long surpassing the limit of sequence models like LSTM, Transformers, etc. We propose to shrink the sequence by only retaining the columns containing entities linked to the statement to alleviate such a memory issue. In order to encode such sub-table as a sequence, we propose two different linearization methods, as is depicted in Figure 4. (i) Concatenation: we simply concatenate the table cells with [SEP] tokens in between and restart position counter at the cell boundaries; the column name is fed as another type embedding to the input layer. Such design retains the table information in its machine format. (ii) Template: we adopt simple natural language templates to transform a table into a "somewhat natural" sentence. Taking the horizontal scan as an example, we linearize a table as "row one's game is 51; the date is February; ..., the score is 3.4 (ot). row 2 is ...". The isolated cells are connected with punctuations and copula verbs in a language-like format.

After obtaining the linearized sub-table $\tilde{\mathbf{T}}$, we concatenate it with the natural language statement S and prefix a [CLS] token to the sentence to obtain the sequence-level representation $H = f_{BERT}([\tilde{\mathbf{T}}, S])$, with $H \in \mathbb{R}^{768}$ from pre-trained BERT (Devlin et al., 2019). The representation is further fed into multi-layer perceptron f_{MLP} to obtain the entailment probability $p_\theta(\tilde{\mathbf{T}}, S) = \sigma(f_{MLP}(H))$, where σ is the sigmoid function. We finetune the model θ (including the parameters of BERT and MLP) to minimize the binary cross entropy $\mathcal{L}(p_\theta(\tilde{\mathbf{T}}, S), L)$ on the training set. At test time, we use the trained BERT model to compute the matching probability between the (table, statement) pair, and classify it as ENTAILED statement when $p_\theta(\tilde{\mathbf{T}}, S)$ is greater than 0.5.